STRING HANDLING

Experiment 1: String Constructors

Problem Statement:

Write a Java program to demonstrate different ways of creating and initializing `String` objects using various **String constructors**.

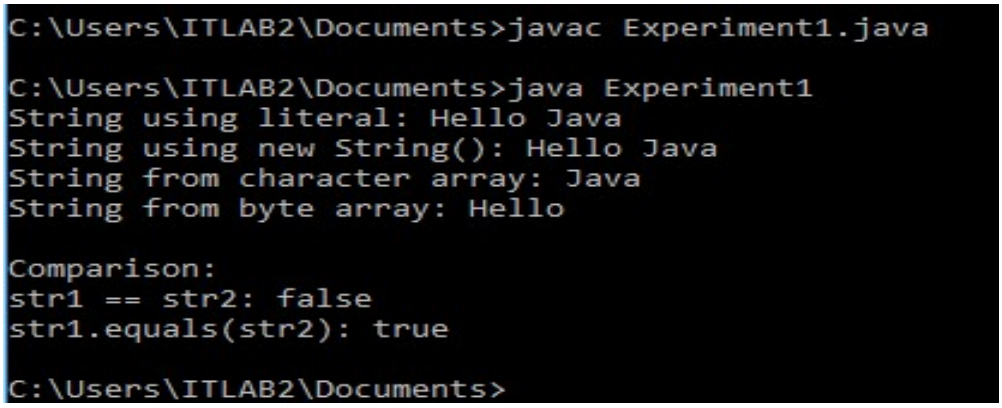
Requirements:

- Create a String using a string literal.
- Create a String using `new String()`.
- Create a String from a character array.
- Create a String from a byte array.
- Display all the created strings.
- Demonstrate the difference between String objects created using literals and `new`.

PROGRAM:

```
public class Experiment1 {
    public static void main(String[] args) {
        String str1 = "Hello Java";
        String str2 = new String("Hello Java");
        char[] chars = {'J', 'a', 'v', 'a'};
        String str3 = new String(chars);
        byte[] bytes = {72, 101, 108, 108, 111};
        String str4 = new String(bytes);
        System.out.println("String using literal: " + str1);
        System.out.println("String using new String(): " + str2);
        System.out.println("String from character array: " + str3);
        System.out.println("String from byte array: " + str4);
        System.out.println("\nComparison:");
        System.out.println("str1 == str2: " + (str1 == str2));
        System.out.println("str1.equals(str2): " + str1.equals(str2));
    }
}
```

OUTPUT:



Experiment 2: StringBuffer Class

Problem Statement:
Write a Java program to demonstrate the functionality of the **StringBuffer** class by performing various mutable string operations.

Requirements:

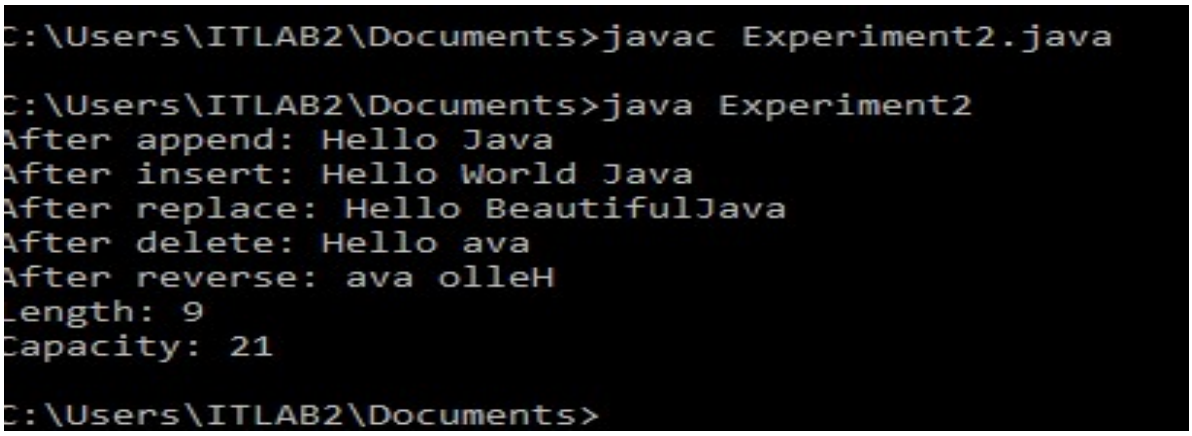
- Create a `StringBuffer` object.
- Append text to the buffer.
- Insert text at a specified position.
- Replace a portion of the string.
- Delete characters from the buffer.
- Reverse the contents.
- Display the current length and capacity of the `StringBuffer`.

PROGRAM:

```
public class Experiment2 {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" Java");
        System.out.println("After append: " + sb);
        sb.insert(6, "World ");
        System.out.println("After insert: " + sb);
        sb.replace(6, 12, "Beautiful");
        System.out.println("After replace: " + sb);
        sb.delete(6, 16);
```

```
        System.out.println("After delete: " + sb);
        sb.reverse();
        System.out.println("After reverse: " + sb);
        System.out.println("Length: " + sb.length());
        System.out.println("Capacity: " + sb.capacity());
    }
}
```

OUTPUT:



Experiment 3: StringTokenizer Class

Problem Statement:
Write a Java program to tokenize a given sentence using the **StringTokenizer** class and display each individual token.

Requirements:

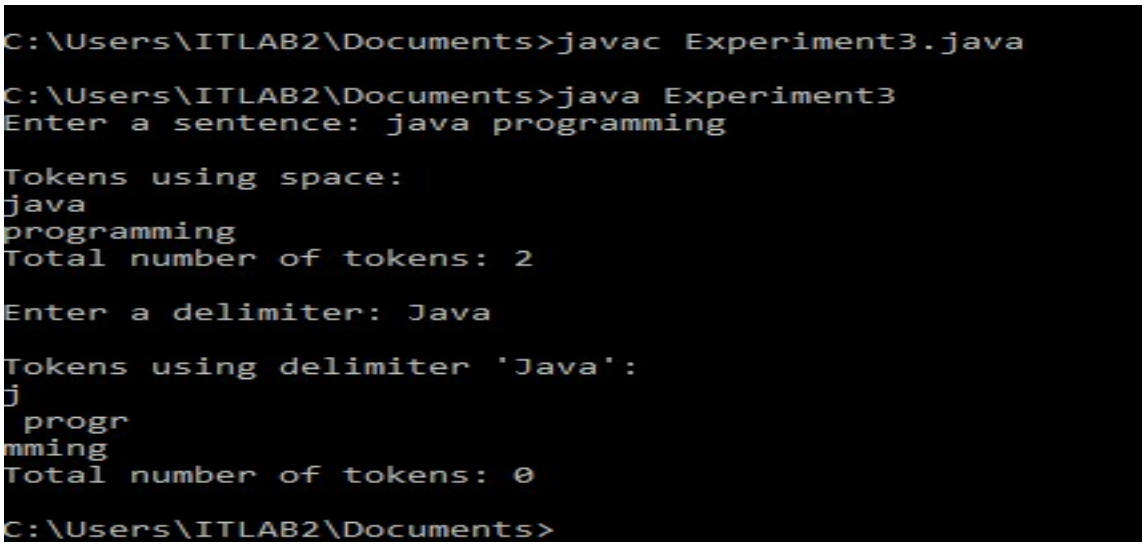
- Read a sentence from the user.
- Use `StringTokenizer` to split the sentence into words.
- Display each token separately.
- Display the total number of tokens.
- Repeat the operation using a user-specified delimiter.

PROGRAM:

```
import java.util.Scanner;
import java.util.StringTokenizer;
public class Experiment3 {
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String sentence = sc.nextLine();
        StringTokenizer st = new StringTokenizer(sentence);
        System.out.println("\nTokens using space:");
        while (st.hasMoreTokens()) {
            System.out.println(st.nextToken());
        }
        st = new StringTokenizer(sentence);
        System.out.println("Total number of tokens: " + st.countTokens());
        System.out.print("\nEnter a delimiter: ");
        String delimiter = sc.nextLine();
        StringTokenizer st2 = new StringTokenizer(sentence, delimiter);
        System.out.println("\nTokens using delimiter '" + delimiter + "':");
        while (st2.hasMoreTokens()) {
            System.out.println(st2.nextToken());
        }
        System.out.println("Total number of tokens: " + st2.countTokens());
        sc.close();
    }
}
```

OUTPUT:



Inheritance

Experiment 4: Basic Inheritance

Problem Statement:
Write a Java program to demonstrate **single inheritance** by creating a superclass containing common properties and methods and a subclass that inherits and extends the functionality of the superclass.

Requirements:

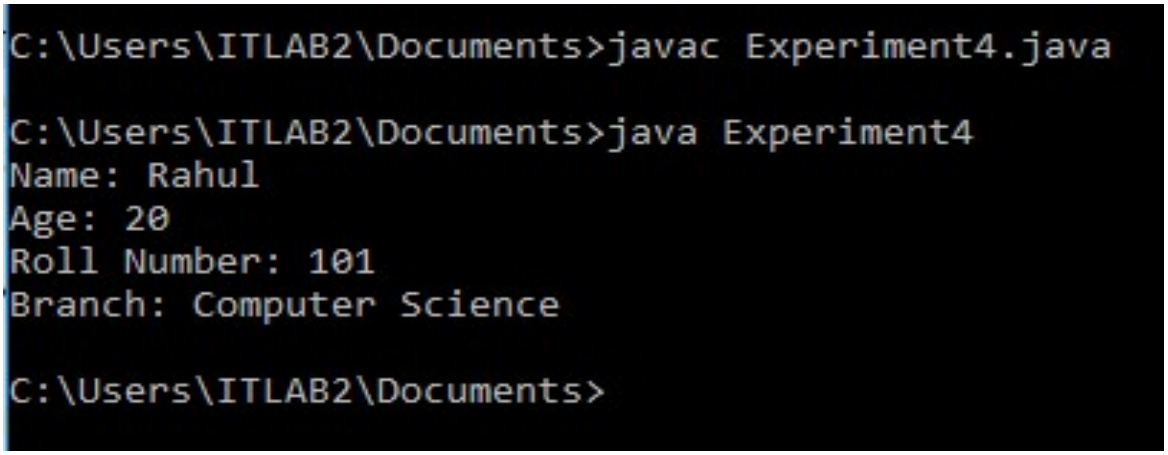
- Create a superclass `Person` with attributes such as name and age.
- Define a method to display the person details.
- Create a subclass `Student` that inherits from `Person`.
- Add student-specific attributes such as roll number and branch.
- Display all details using the subclass object.

PROGRAM:

```
class Person {
    String name;
    int age;
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    void displayPersonDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
class Student extends Person {
    int rollNumber;
    String branch;
    Student(String name, int age, int rollNumber, String branch) {
        super(name, age);
        this.rollNumber = rollNumber;
        this.branch = branch;
```

```
    }
    void displayStudentDetails() {
        displayPersonDetails();
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Branch: " + branch);
    }
}
public class Experiment4 {
    public static void main(String[] args) {
        Student student = new Student(
            "Rahul",
            20,
            101,
            "Computer Science"
        );
        student.displayStudentDetails();
    }
}
```

OUTPUT:



Experiment 5: Using `super` Keyword

Problem Statement:
Write a Java program to demonstrate the use of the `super` **keyword** for accessing superclass variables, methods, and constructors.

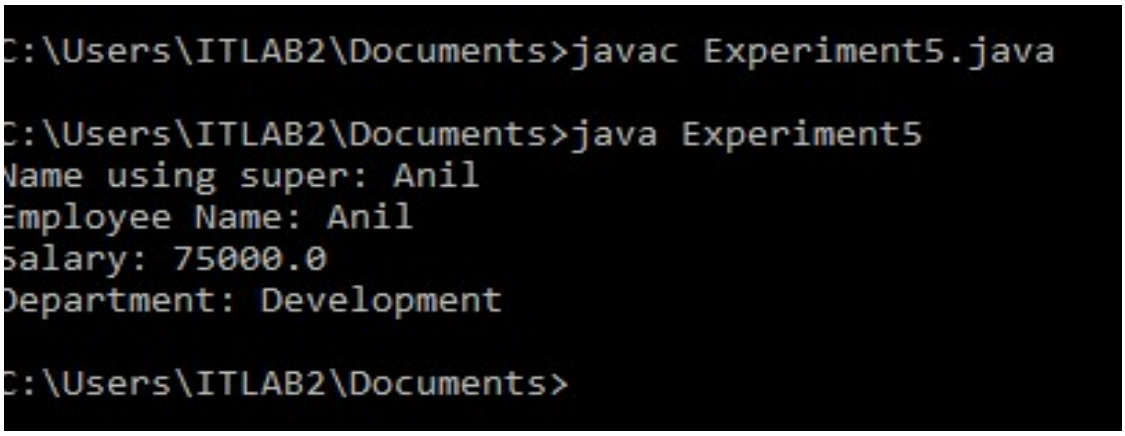
Requirements:

- Create a superclass `Employee` with appropriate variables and methods.
- Create a subclass `Manager`.
- Use `super` to access a superclass variable.
- Use `super` to invoke a superclass method.
- Use `super()` to invoke the superclass constructor.
- Display the complete employee and manager details.

PROGRAM:

```
class Employee {
   String name;
   double salary;
   Employee(String name, double salary) {
      this.name = name;
      this.salary = salary;
   }
   void displayDetails() {
      System.out.println("Employee Name: " + name);
      System.out.println("Salary: " + salary);
   }
}
class Manager extends Employee {
   String department;
   Manager(String name, double salary, String department) {
      super(name, salary);
      this.department = department;
   }
   void displayManagerDetails() {
      System.out.println("Name using super: " + super.name);
      super.displayDetails();
      System.out.println("Department: " + department);
   }
}
public class Experiment5 {
   public static void main(String[] args) {
      Manager manager = new Manager(
         "Anil",
         75000,
         "Development"
      );
      manager.displayManagerDetails();
   }
}
```

OUTPUT:


C:\Users\ITLAB2\Documents>javac Experiment5.java
C:\Users\ITLAB2\Documents>java Experiment5
Name using super: Anil
Employee Name: Anil
Salary: 75000.0
Department: Development
C:\Users\ITLAB2\Documents>